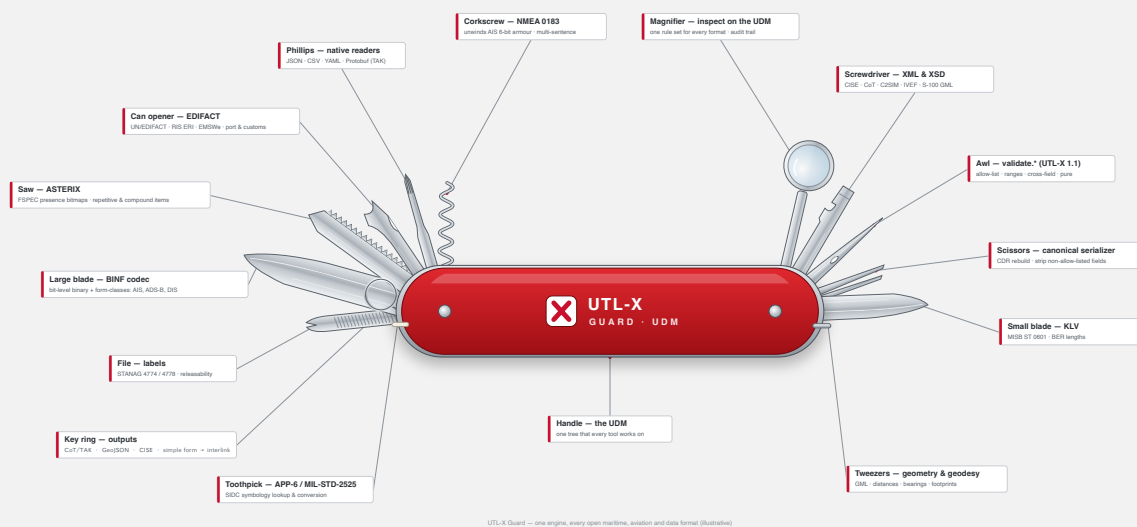


UTLX Guard

The Civilian Edition



A content guard for open standards
rebuild what is clean, reject what is wrong — every format, one rule set

Ir. Marcel A. Grauwen

UTL-X Guard — The Civilian Edition

Copyright © 2026 Ir. Marcel A. Grauven. All rights reserved.

Published by GLOMIDCO B.V., The Netherlands. Exploratory edition, 2026.

UTL-X is open-source software (AGPL-3.0), freely available at <https://github.com/grauven/utl-x>. This book depends on **UTL-X 1.1** (the `validate.*` semantic-validation extension).

This is the civilian companion to *UTL-X MIL — The Content Guard* (the defence edition). The two share one engine and one architecture; the MIL edition adds restricted-format packs and higher assurance — not a different design. See also *UTL-X: One Language, All Formats* and *UTLXe on Azure*.

Status — exploratory. This book is a capability and go-to-market study of a civilian, open-standards content guard built on UTL-X. It is not a product specification, nor legal, export-control or accreditation advice. External standards are named indicatively; verify against primary sources.

Typeset with Typst in New Computer Modern.

Table of Contents

1	The UTL-X Guard	7
1.1	The problem it ends	7
1.2	The six non-negotiables	7
1.3	How to read this book	8
2	What It Does, and Why It's Different	9
2.1	What a guard does	9
2.2	Always rebuilt — and in any format	9
2.3	The leverage: one rule set for every format	10
2.4	It is not a WAF	10
2.5	The gap it fills	11
2.6	Five differentiators	11
3	Where It Fits — Markets and Deployment	12
3.1	Markets and first customers	12
3.2	Four deployment shapes	12
3.3	Getting messages in and out	12
3.4	Positioning	13
4	Open Standards — What It Speaks	14
4.1	The format scope	14
4.2	Dual-use ground, already covered	14
4.3	What is deliberately out of scope	14
4.4	Open core	14
5	One Language, One Model	16
5.1	The shape of a transformation	16
5.2	The Universal Data Model	16
5.3	Three accessors: data, attribute, metadata	16
5.4	The contract — four guarantees	16
6	Every Format Becomes One Tree	18
6.1	The problem with binary: it does not describe itself	18
6.2	Form-class: the layout as data	18
6.3	The encoding spectrum, one codec	18
6.4	Why generic, not “an AIS reader”	18
6.5	For the guard, this matters twice	18
7	Validation, and Safe by Construction	19
7.1	Validation as a first-class citizen — UTL-X 1.1	19
7.2	The parser is the crux	19
7.3	The hardened “guard profile” parser	19
7.4	The anti-round-trip serializer	20
8	Rules, Verdicts, and Coverage	21
8.1	Seven categories of rule	21
8.2	Labels — STANAG 4774 / 4778	21
8.3	The verdict policy — rebuild what is clean, reject what is wrong	21
8.4	Why not “repair and pass”?	22
8.5	What happens to a rejected message	22
8.6	Three verdict states	22
8.7	OWASP — a coverage map, not a rule port	22
8.8	NIST 800-53 AC-4 — a control mapping	23
9	Assurance, Deployment, and the Path to Market	25
9.1	Assurance — climb in evidence first, then credentials	25
9.2	Hardware — mostly none	25

9.3 Roadmap	27
9.4 Legal, licensing, export	27
9.5 Bottom line	28
10 Proof of Concept — the Split Guard	29
10.1 The demonstrator: a split guard on two boards	29
10.2 Architecture	30
10.3 What it demonstrates	31
10.4 Success criteria (indicative)	31
10.5 Scope and effort	32
10.6 What this PoC deliberately does NOT do	32
10.7 The investment case (civilian)	32
10.8 Next steps	32
11 Appendix A: UTL-X Quick Reference	33
11.1 File structure	33
11.2 Format headers	33
11.3 Access	33
11.4 Validation (1.1)	33
11.5 The guard, in one line	33
11.6 The non-negotiables	33
11.7 The verdict policy	33
12 Appendix B: Open-Format Catalogue	34
12.1 Text-structured → native readers (mapping only)	34
12.2 Schema / metadata → native	34
12.3 Binary → BINF codec + form-class	34
12.4 EDI / delimited → EDIFACT-class reader	34
12.5 Code systems / labels	34
12.6 Out of scope (MIL edition)	34
13 Appendix C: Sources & Further Reading	35
13.1 UTL-X design documents	35
13.2 Companion books	35
13.3 External standards & guidance (verify against primary sources)	35
13.4 Cross-domain literature & standards basis	35

PART I

The Guard

What it is, what it does, where it fits, and the formats it speaks.

1 The UTL-X Guard

The UTL-X Guard is a **content control point for the boundary between two zones of different trust** — two agencies, an OT network and its enterprise, a ministry and its chain partners, a coast-guard system and a coalition picture. It inspects every message that tries to cross, passes only what policy explicitly allows, and rebuilds it clean. It does this for **any** open message format — JSON, XML, CSV, EDIFACT, and binary standards such as AIS, ADS-B, ASTERIX and KLV — using **one** set of rules.

This is the **civilian edition**: open standards only, software-first, sold and fielded now. It is the same engine, the same architecture and the same non-negotiables as the defence edition (*UTL-X MIL — The Content Guard*); the MIL edition adds restricted-format packs and higher assurance on top — not a different product.

At a glance

What it is	a declarative, auditable content guard (the civil term is Content Disarm & Reconstruction)
How it works	every message is parsed to one model and rebuilt — never passed through — then re-serialized to the same format or a different one
What is new	one canonical model inspects every format — text and binary — with one rule set, instead of a separate guard per format
Built on	UTL-X, an open-source (AGPL-3.0), format-agnostic transformation language — pure, deterministic, safe on a boundary
Who it is for	coast guard / CISE, critical-infrastructure OT, government data sharing, air-traffic surveillance, police and crisis response
Connects via	any message bus or endpoint — Kafka, AMQP, MQTT, DDS, files — through the Dapr component model; one-way, fire-and-forget
Fielding	software gateway, diode-adjacent filter, or a split two-owner guard — on ordinary servers; hardware only where assurance or OT require it
Status	exploratory capability + go-to-market study; not an accredited product

1.1 The problem it ends

When two zones of different trust must exchange messages, something has to look at each message and decide whether it may cross — and rebuild it so nothing unexpected rides along. The trouble is that every message format has needed its **own** inspector: a JSON guard, an XML guard, a guard for each binary feed. Each is a separate piece of software, a separate attack surface, and a separate rule set to write, test and assure. The real cost is the **multiplication** — N formats means N inspectors and N ways to be wrong.

UTL-X parses every format it reads — text or binary — into a single internal model, the Universal Data Model (UDM). Write the inspection rules against the UDM, and the picture collapses:

N formats → one canonical model → one rule set → one re-serializer

1.2 The six non-negotiables

“Civilian” must never mean “softer”. These six rules are exactly what make the later step to MIL a matter of **assurance and packs**, not redesign:

No pass-through	every message is parsed to the UDM and re-serialized; the original bytes never reach egress — even an approved message is rebuilt, not forwarded
------------------------	--

Allow-list, fail-closed	only content matching an approved Message Contract and rule set passes; everything else is blocked
Pure and deterministic	only UTL-X 1.0 / 1.1 in the trusted path; the probabilistic ai.* (2.0) is excluded
Canonical, low-fidelity output	comments, BOMs, padding, unknown and non-allow-listed fields are dropped on rebuild
Every verdict auditable	input hash, output hash, contract ID, rule-set version, rule IDs, timestamp — a reproducible record
UDM only sees what it models	opaque binary payloads are blocked or handed to a specialised inspector, never waved through

Chapter 8 turns these into the operational **verdict policy** — rebuild what is clean, reject what is wrong, never repair.

1.3 How to read this book

Part I is the product: what the guard is (this chapter), what it does and why it is different (Chapter 2), where it fits — markets and deployment (Chapter 3), and the open formats it speaks (Chapter 4). **Part II** is the mechanism for the technical evaluator: the language and model, binary decoding, validation and hardening, and the rules and verdict policy (Chapters 5–8). **Part III** is fielding it: the assurance route, deployment and the path to market (Chapter 9), and a concrete, buildable proof of concept with the civilian investment case (Chapter 10).

2 What It Does, and Why It's Different

Chapter 1 made the promise; this chapter is the value proposition — what the guard actually does, and the handful of properties that set it apart from everything else on a boundary.

2.1 What a guard does

A guard sits between two zones of different trust and does three things, in order:

normalise untrusted input → inspect against policy → pass only what is explicitly allowed, rebuilt clean

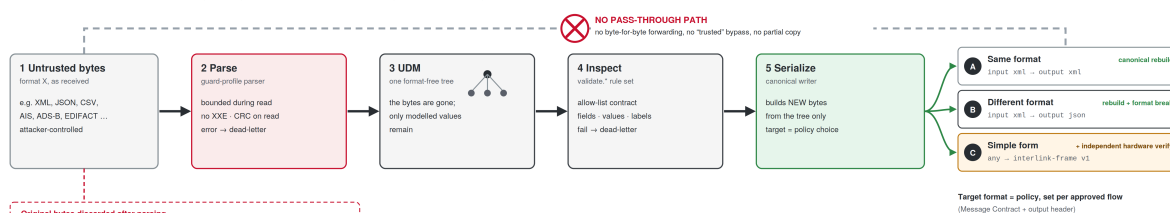
There is no pass-through path: even an approved message is never forwarded as received — it is rebuilt from the UDM, and the original bytes never reach egress. The civil world calls this shape Content Disarm & Reconstruction (CDR). The UTL-X Guard is not a new product built from scratch to do it — it is this shape expressed in a transformation engine: parse to the UDM, inspect with rules, re-serialize a clean canonical message. Rebuilding from the model is itself a disarming step (Chapter 7).

2.2 Always rebuilt — and in any format

From the UDM tree the guard can re-serialize **to the same format** (a clean, canonical drop-in replacement) or **to a different format** (decode a binary AIS or ADS-B frame and emit JSON or XML for a modern consumer). Changing the format is itself a defence — a **format break**: an exploit crafted for the input encoding cannot survive being rebuilt into a different one. And it comes for free as an **integration** capability: the same box that guards the boundary also bridges formats.

Every message is rebuilt — never forwarded

Parse → UDM → inspect → serialize is mandatory for every message. The target format is a policy choice; the original bytes never reach egress.



Example — one message, three policy choices

Struck-through parts arrive but are never re-emitted: they are not in the UDM, or not on the allow-list.



Disarm strength grows with the policy choice



Note: a format break defeats encoding-level attacks. Hostile content that survives semantically — e.g. a malicious URL in a text field — is the job of the inspection rules (value, keyword and field allow-list), not of the serializer.

Figure — Mandatory rebuild: every message is parsed to the UDM, inspected, and serialized anew; only the target format varies by policy (illustrative).

Figure 1: Every message is rebuilt, never forwarded. There is no pass-through path (top): untrusted bytes are parsed to the UDM, inspected, and a fresh message serialized; the original bytes are discarded after parsing and kept only as a hash. The target format is a policy choice — same format, a different format (plus a format break), or a simple form for independent verification. Disarm strength grows with the choice (bottom); a format break stops encoding attacks, not hostile content that survives semantically — that is the rules' job (Chapters 7, 8).

2.3 The leverage: one rule set for every format

The difference that matters commercially is the collapse from many inspectors to one. Because every format normalises into one UDM, a releasability check, a value-range check, a forbidden-field check is each written **once** and applied to a JSON document and a binary AIS frame alike.

This is, deliberately, a **separation of layers**: the readers and serializers deal with **formats**; the policy lives one layer up, **on the UDM**, and never names a format at all. A new format is a new **reader**, never a new rule; a new rule applies to **every** format at once. The single policy a reviewer accredits is the same policy that runs on every input — the heart of the assurance argument.

UTL-X MIL Guard — from any message format to one tree, inspected, and back

Every input is parsed into the Universal Data Model (UDM); one rule set inspects the tree; a canonical serializer rebuilds only what is allowed.

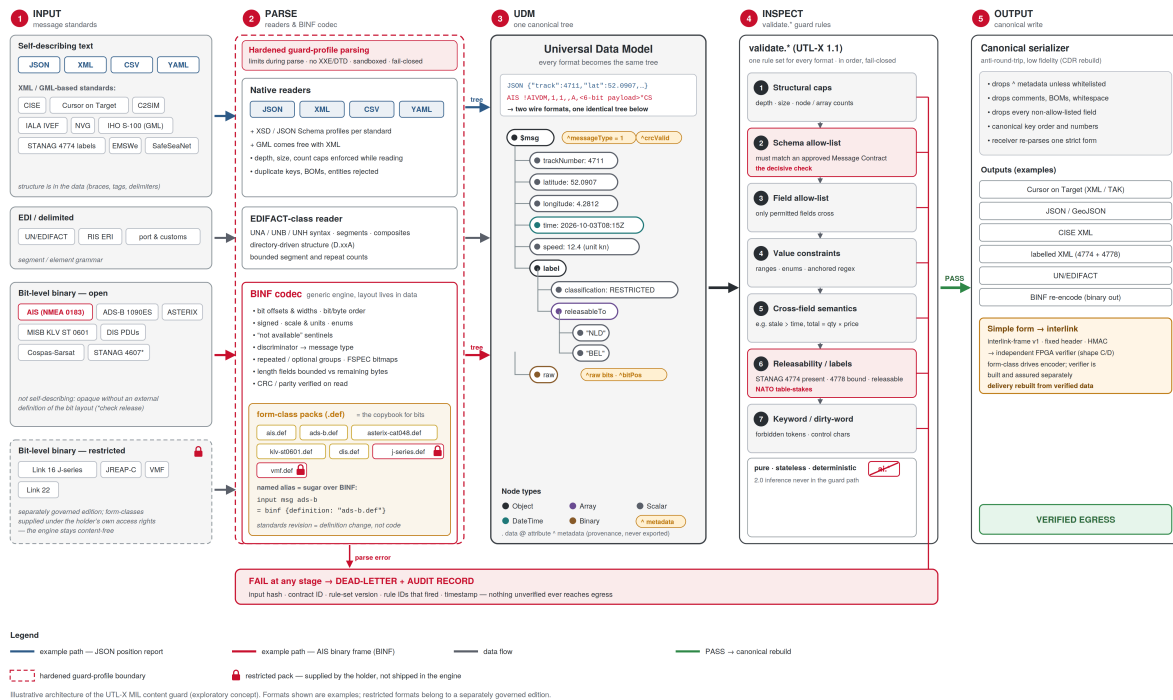


Figure 2: From any format to one tree: inputs of every encoding class are parsed (native readers, an EDIFACT reader, or the BINF codec with a form-class) into the UDM; one pure `validate.*` rule set inspects the tree; a canonical serializer rebuilds only what is allowed. Any failure goes to isolation with an audit record.

2.4 It is not a WAF

A web application firewall inspects HTTP traffic for known-bad patterns — a **deny-list**, leaky by design. A content guard passes only **known-good** content — an **allow-list**, fail-closed — and decides about **data**, not network traffic.

WAF / API gateway	deny-list · HTTP · blocks known-bad · leaks the unknown
UTL-X Guard	allow-list · any format · passes only approved content, rebuilt · fail-closed

This is an **established classification**, not our own claim. In the cross-domain taxonomy of US DoDI 8540.01 (adopted by the NCDSMO) — *access, transfer, multilevel* — the guard is a **transfer** solution: message-oriented, store-and-forward, one verdict per message, in the tradition of the high-assurance **mail guard** (Smith, 1994), which — unlike a firewall — assured its control **even under attack and failure**. Where two-way traffic is needed it is built as two separate one-way flows, never a session through the guard. Chapter 8 maps the pipeline to the NIST 800-53 AC-4 control enhancements.

2.5 The gap it fills

*File-CDR products disarm **documents**. API gateways and WAFs **route and filter** traffic. Diodes enforce **direction**. Nobody rebuilds **structured messages** across every format — text and binary — under one declarative, auditable rule set.*

2.6 Five differentiators

Every format, one model	text and binary — JSON, XML, CSV, EDIFACT, AIS, ADS-B, ASTERIX, KLV — all inspected by one rule set on the UDM
Allow-list, fail-closed	passes only approved content; blocks everything else, including novel attacks a deny-list would miss
Rebuilt, not forwarded	the received bytes never cross; what crosses is a clean canonical message built only from what the rules allowed
Declarative and auditable	rules are readable by security staff and traceable to policy; every verdict is reproducible — an assessment asset
Open-core and dual-use	open-source engine and public packs; the same engine serves civilian data-sharing and, with restricted packs, defence crossings — “MIL-ready by design”

Chapter 3 shows where this is actually fielded — the civilian markets and the deployment shapes.

3 Where It Fits — Markets and Deployment

The civilian guard depends on nothing Glomidco does not control — no sponsor, no export-control clearance, no multi-year accreditation. That is the whole point: it can be **sold, fielded and proven now**, while the MIL edition becomes “the civilian guard + restricted packs + higher assurance”.

3.1 Markets and first customers

Coast guard & maritime	a CISE adaptor that also enforces policy — sharing tracks and incidents between agencies and countries. CISE needs an adaptor per connected system; a guard is an adaptor with policy. Formats: CISE, AIS, IVEF, EDIFACT.
Critical infrastructure / OT	one-way export of sensor/SCADA-derived data out of a control network; controlled import of updates and manifests. Diodes are common here, and NIS2 pushes demonstrable control of data flows. Formats: CSV, JSON, XML, historian exports, EDIFACT.
Government data sharing	ministries and chain partners exchanging structured data across trust zones — a natural market for a national product assessment. Formats: JSON, XML, OData, XSD/JSON-Schema, labels.
Air-traffic / airport surveillance	filtering and normalising surveillance feeds between operators — a strong binary showcase for conservative buyers. Formats: ASTERIX, ADS-B.
Police / border / crisis response	drone metadata and situational awareness shared between agencies — the dual-use stepping stone toward defence. Formats: MISB KLV, Cursor-on-Target, C2SIM.

The sensible go-to-market is **one wedge**: pick one of the first three segments for the first reference customer, and let that customer pull the formats and the first credential forward.

3.2 Four deployment shapes

A — content gateway	an inline gateway or API proxy between two zones or organisations; carries the allow-list, schema, label and content policy. The first and simplest product.
B — diode-adjacent filter	content policy beside a hardware data diode (OT, low→high import); the diode gives one-way flow assurance, UTL-X gives content assurance. Partner with diode vendors.
D — split guard	two owners, two halves, one per zone, each with its own policy, meeting only over a one-way interlink — for chain partners who will not host each other’s box.
C — hardsec transform stage	UTL-X transforms complex input into a simple form a separate hardware verifier checks; mostly a high-assurance / MIL play. Keep the hook; field it with a partner later.

Shapes A, B and D are the civilian product; C is where civilian meets high assurance.

3.3 Getting messages in and out

The guard decides on **content**, not on the wire a message arrived on — so the transport is a configuration choice, not a redesign. Reusing UTL-X’s Dapr component model, it connects to message buses and brokers (Kafka, Pulsar, AMQP/RabbitMQ, MQTT, JMS, cloud queues), real-time pub/sub (OMG **DDS**), and

direct or batch endpoints (TCP/UDP, HTTP/REST, gRPC, file drops). Two properties hold throughout: the ingress transport is **terminated, not tunnelled** (a protocol break — no session state crosses; a fresh message is published on the receiver’s independent bus), and delivery is a **fire-and-forget forward** — consume, decide, forward a rebuilt message or route to isolation, with no synchronous reply path back across the boundary.

3.4 Positioning

File CDR (OPSWAT, Glasswall, Votiro)	complementary — files vs messages; integrate for opaque payloads
API gateway / WAF (Kong, Apigee, Azure APIM+WAF, F5)	the “good enough” competitor; UTL-X is allow-list content re-build , not deny-list traffic filtering
Data diodes (Fox Crypto, Arbit, Owl, Waterfall, Advenica)	partners for shape B; some already integrate third-party filters
Integration platforms (ESB / iPaaS)	adjacent — they transform, but are not built as fail-closed guards
Cross-domain vendors (Infodas/Airbus, Everfox, Isode, BAE)	mostly MIL/government high-assurance; meet them on the MIL track

Naming and tagline: **UTL-X Guard** (civilian, open core) and **UTL-X MIL Guard** (restricted packs, accredited) — “MIL-ready by design, proven in civilian service.”

4 Open Standards — What It Speaks

The civilian edition is defined by a line drawn around **open standards only**. Everything it reads is publicly specified and buildable without a sponsor or restricted access. That line is what lets the product ship now; the restricted tactical data links stay in the separately governed MIL edition.

4.1 The format scope

Data (Tier 1)	JSON, XML, CSV, YAML, OData — native readers, already in UTL-X 1.0
Schema / metadata (Tier 2)	XSD, JSON Schema, Avro, Protobuf, OData EDMX, Table Schema — native
Maritime / coast guard	CISE, AIS (NMEA 0183), IALA IVEF, IHO S-100 (GML products), EMSWe, Cospas-Sarsat
Aviation / surveillance	ADS-B (1090ES), ASTERIX — binary, via the form-class mechanism
Ports, logistics, inland shipping	UN/EDIFACT, RIS (ERI, NtS, Inland AIS)
Open defence / security (dual-use)	MISB KLV ST 0601, Cursor-on-Target / TAK, C2SIM, DIS, APP-6 / MIL-STD-2525 symbology
Labels	STANAG 4774 / 4778 (public), national government classification labels — XML + <code>validate.*</code>

The encoding, not the domain, decides the work. Text-structured formats (the XML and JSON families) are already read and written by UTL-X, so for them the work is **mapping**, not decoding. Binary formats (AIS, ADS-B, ASTERIX, KLV) go through the BINf codec and a **form-class** (Chapter 6). A dedicated EDIFACT-class reader covers the delimited dialects.

4.2 Dual-use ground, already covered

The open “military” formats — KLV, Cursor-on-Target, C2SIM, DIS — are **publicly available**. So the civilian guard already covers substantial dual-use ground without touching anything restricted: police and border-security drones, civil–military cooperation in crisis response, training and simulation. This is the bridge toward defence that needs no defence paperwork.

4.3 What is deliberately out of scope

Restricted tactical data links	Link 16 J-series, JREAP-C, VMF, Link 22 — the MIL edition’s separately governed packs
Unclear release status	anything whose distribution terms are unverified (e.g. STANAG 4607 until checked)

These are not a different architecture — they are **packs** the MIL edition adds under its own access rights. The civilian engine stays content-free: restricted form-classes never enter the open repository.

4.4 Open core

The split falls out cleanly as an open-core model: the **engine** and the **public packs** are open source (AGPL-3.0); guard-specific productisation — the contract/policy tooling, certified builds, monitoring and support — is the commercial layer; restricted packs are the MIL edition, supplied separately. Nothing the civilian guard builds is throw-away: the engine, parser profile, rule library, canonical serializer and audit trail are identical across editions.

PART II

How It Works

One language, one model, binary decoding, validation, hardening, and the verdict.

5 One Language, One Model

This chapter is a short tour of UTL-X — enough to read every example in this book — and the contract that makes the language safe to run on a boundary. For the full reference, see *UTL-X: One Language, All Formats*.

5.1 The shape of a transformation

Every UTL-X program has the same three-part shape: a header, a --- separator, and a single body expression.

```
%utlx 1.1
input  json
output json
---
{ /* one functional expression that is the transformation */ }
```

The header declares the language version and the input/output formats (with options, or a **form-class** for binary). After --- comes one functional expression — no statements, no mutation, no imperative loops.

5.2 The Universal Data Model

The single most important decision in UTL-X is that **all** inputs are parsed into one internal model — the Universal Data Model (UDM) — before any expression runs, and all outputs are serialized from it. Format in, UDM, format out. The body never sees JSON or XML or bytes; it sees a tree.

Scalar	string, number, boolean, null
Object	named children (a JSON object, an XML element)
Array	indexed children (a JSON array, repeating XML)
Binary	raw bytes (a leaf)
DateTime	first-class temporal values

Because every format collapses to these few node types, a **rule** written for the UDM polices every format on one model — the guard of Chapter 2.

5.3 Three accessors: data, attribute, metadata

\$input.name — .	data / content — the ordinary case
\$input.El.@id — @	an attribute (e.g. an XML attribute)
\$input.x^crcValid — ^	metadata about a node: provenance, decode results, the security label

The ^ metadata channel is where a binary decoder records CRC validity or the raw bytes, and where a label (STANAG 4774) rides alongside the data — present for policy, never silently promoted into the data.

5.4 The contract — four guarantees

The guard rests on four guarantees UTL-X makes about **every** program:

Pure	the same input always produces the same output — no side effects, no clock, no network
Stateless	no memory between messages; the thousandth maps exactly as the first
Deterministic	no probability, no randomness, no model weights — so a verdict is reproducible and reviewable

Single-pass	one input (or N correlated inputs) produces one output in bounded time and memory
--------------------	---

These are what make a UTL-X program safe to run **inline on a trust boundary** — in process, reviewable, reproducible. A reviewer can read a transformation and know exactly what it will do with any input, because it cannot do anything else. Chapter 7 adds validation (`validate.*`) while keeping all four intact — and draws the line the guard depends on: the probabilistic `ai.*` of UTL-X 2.0 **breaks** purity and determinism, and is therefore forbidden anywhere inside a guard.

6 Every Format Becomes One Tree

The UDM of Chapter 5 is only useful if every format can reach it. For text-structured formats that is easy — UTL-X already reads XML and JSON. The interesting case is **binary**: an AIS burst, an ADS-B squitter, an ASTERIX record — bits packed with no delimiters and no tags.

6.1 The problem with binary: it does not describe itself

A JSON document carries its own structure — braces, keys, values. A bit-packed frame does not. An ADS-B 1090ES message is 112 bits with a 24-bit CRC; nothing **in** those bits says “the next 5 bits are a type code”. The layout lives **outside** the data, in the specification — so the engine must be given it.

6.2 Form-class: the layout as data

The oldest name for a non-self-describing record whose shape is defined externally comes from the trading floor: a **form-class**. UTL-X adopts the term. A form-class is a **definition** — a declarative description of a binary format’s physical layout: fields, bit offsets and widths, encodings, scalings, enumerations and discriminators. It is **data**, not code.

```
%utlx 1.1
input  binf { definition: "packs/ais.def" }
output json
---
{ mmsi: $input.mmsi, lat: $input.lat, lon: $input.lon }
```

The **BINF codec** reads the raw bytes, consults the form-class, and produces a UDM tree — the same kind of tree a JSON reader produces. From that point on the guard is format-agnostic: it inspects a tree, indifferent to the fact that the tree came from bits.

raw bytes + form-class → BINF codec → UDM tree → one rule set

6.3 The encoding spectrum, one codec

Bit-level	fields at arbitrary bit offsets, no delimiters — AIS, ADS-B
Byte-structured	records / PDUs on byte boundaries — DIS PDUs
Self-tagged (KLV)	key-length-value, partly self-describing — MISB KLV ST 0601
Presence-driven	existence bitmaps declaring which optional fields are present — ASTERIX FSPEC

The presence-driven case (ASTERIX FSPEC) is the demanding one, and the open ASTERIX pack is the proving ground for it.

6.4 Why generic, not “an AIS reader”

Baking each binary format into the engine as a named reader would mean a code change per format and a content-bearing engine. Making the engine a **generic bit-level codec** driven by a form-class means a new format is a new **definition file** — buildable by whoever holds the spec, with no engine change. Named aliases are then a convenience on top: `input ads-b` can resolve to `binf { definition: "packs/ads-b.def"`

`}` so day-to-day scripts read cleanly, while the engine stays generic underneath.

6.5 For the guard, this matters twice

First, the same rule set polices a binary AIS frame exactly as it polices a JSON document — because by the time the rules run, both are the same tree. Second, because a form-class describes a layout, BINF reads **and writes** it: the guard can re-serialize a decoded binary frame as a canonical binary message, or as clean JSON — the format break of Chapter 2, and a free integration bridge.

7 Validation, and Safe by Construction

Two things make the guard trustworthy: it can **assert** that content is correct before passing it (validation), and it survives parsing hostile input to do so (hardening). This chapter covers both.

7.1 Validation as a first-class citizen — UTL-X 1.1

UTL-X 1.0 maps and restructures data; it does not **validate** it. UTL-X 1.1 closes that gap with the `validate.*` namespace, which asserts facts over the UDM and returns a structured `ValidationResult` — which rule failed, on which field, and why — suitable for routing and for an audit trail:

```
validate.inRange($msg.latitude, -90, 90)
validate.oneOf($msg.status, ["OK", "WARN", "FAIL"])
validate.conformsTo($msg, "contracts/approved/position-report:1.0.0")
validate.assert($msg.stale > $msg.time, "stale must be after time")
```

Crucially, `validate.*` keeps **every** 1.0 guarantee — pure, stateless, deterministic — so it only adds; a 1.0 script runs unchanged on a 1.1 engine. That is exactly why it is a **minor** version, and why it is the natural rule engine for a guard: a guard’s verdict must be reproducible and reviewable, and `validate.*` is precisely a deterministic rule layer. The probabilistic `ai.*` of 2.0 is the opposite, and is excluded from the trusted path.

7.2 The parser is the crux

A content guard must parse untrusted, possibly hostile input **before** it can inspect anything. The parser is the single most dangerous component in the design, because it touches the attacker’s bytes first. The good news: the risk is manageable, and the act of parsing-then-rebuilding is itself a defence.

Resource bombs	entity expansion (billion laughs), deep nesting, huge arrays/strings
Length-field overflow	a 32-bit length claiming 4 GB; attacker-controlled repeat counts in binary
XXE / external entities	DTD and external-entity resolution in XML
Parser differential	the guard’s parser and the receiver’s disagree on the same bytes (duplicate keys, BOMs, number precision)

Three properties turn “the parser is dangerous” into “the parser is a bounded risk”:

Memory-safe runtime	UTL-X runs on the JVM / GraalVM; the buffer-overflow-to-code-execution path is largely off the table — the realistic blast radius is denial of service, not attacker code
Parse → rebuild disarms	rebuilding from the UDM drops comments, duplicate keys, smuggled trailing bytes, polyglot tricks — the reconstruction half of CDR, almost for free
Safe parse, then decide	if the parser cannot be exploited, a rogue message is just a well-formed UDM tree the rules inspect — the security decision happens at inspection , not at parsing

7.3 The hardened “guard profile” parser

An ordinary mapping parser is lenient; a guard parser is strict, bounded and fail-closed. The guard profile enforces resource limits **during** parse (size, depth, element count, time, memory) so a bomb dies while being read; disables XXE / DTD / external entities; for binary, bounds and cross-checks every length field and verifies CRC on read; fails closed; runs in a resource-capped sandbox; and is fuzzed as an assurance-grade component (coverage-guided, grammar-based, and differential against the receiving parser).

7.4 The anti-round-trip serializer

Here the guard deliberately does the opposite of ordinary UTL-X. Elsewhere the engine preserves format-fidelity metadata for byte-exact round-trip; a guard wants the reverse — a **low-fidelity, canonical** re-serialization that drops everything non-essential (comments, whitespace, BOMs, non-canonical numbers, and any field not on the allow-list), with keys in canonical order. This disarms the message and defeats parser-differential smuggling, because the receiver re-parses one strict canonical form.

The honest limit: a canonical rebuild and a format break stop attacks aimed at the **encoding**, but not hostile content that survives **semantically** — a malicious URL in a text field, an out-of-range coordinate, a forbidden label. Catching those is the job of the rules, which Chapter 8 — and the verdict policy — makes precise.

8 Rules, Verdicts, and Coverage

The parser (Chapter 7) delivers a safe UDM tree; this chapter is about what the guard then **decides** — the rules, the labels, and above all the **verdict policy** that governs what passes, what is stripped, and what is rejected.

8.1 Seven categories of rule

A guard policy is composed from seven categories of pure **validate**, * rule over the UDM, applied cheapest and most decisive first, fail-closed throughout:

1 — Structural caps	depth, size, node and array counts — anti-bomb, defence-in-depth with the parser
2 — Schema allow-list	the message must conform to an approved contract, with no additional properties
3 — Field allow-list	even within the schema, only permitted fields may cross
4 — Value constraints	ranges, enumerations, anchored (ReDoS-safe) regex, finite numbers
5 — Cross-field semantics	mutual consistency: <code>stale > time</code> , <code>lineTotal == qty × price</code>
6 — Releasability / labelling	classification $\leq$ X, releasable-to the target zone (labels below)
7 — Keyword / dirty-word	forbidden tokens, control characters, embedded markup in free text

The decisive category is the schema allow-list: the message must be an **expected**, **approved** type, or it is rejected. The guard never enumerates bad shapes; it enumerates the single good one.

8.2 Labels — STANAG 4774 / 4778

A 4774 confidentiality label is XML-structured data, so it parses into the UDM like anything else; releasability is then a **pure rule** over the label subtree and the target zone’s clearance. Binding verification (4778) is deterministic — a hash and signature check. Unlabelled or forged-label traffic fails closed. The public 4774/4778 standards are in scope for the civilian edition; national government classification labels are handled the same way.

8.3 The verdict policy — rebuild what is clean, reject what is wrong

This is the rule that governs everything the guard does with a message:

Rebuild what is clean, reject what is wrong — never repair.

A guard must **never** “fish out” the malicious parts and pass the rest. As soon as a message violates policy, the **whole message** goes to isolation (dead-letter or quarantine), with an audit record and an alert. But not everything the guard drops is “malicious” — three categories must be kept distinct:

1 — For- matting noise	comments, whitespace, BOM, odd-but-valid encodings, number formatting, ^ metadata	dropped silently by the canonical serializer ; the message passes. Not fishing out malice — the normal rebuild.
2 — Fields re- moved by pol- icy	internal / high-side-only fields (releasability), debug fields	only if explicitly listed in the contract as “strip” ; the message passes, and the audit records which fields were removed

3 — Violations	parse error, bomb, field off the allow-list (no strip rule), value out of range, wrong / missing / forged label, dirty word, CRC or signature error	reject the whole message → isolation + audit record + alert
-----------------------	---	--

The default for unknown fields is **reject**. “Strip” is an explicit, per-field choice in the Message Contract, never a catch-all.

8.4 Why not “repair and pass”?

Repair means guessing intent	you do not know what the attacker was after, so you cannot know which remainder is safe
Repair logic is an attack surface	an attacker crafts the message so that what is left after cleaning is exactly what they wanted delivered
Meaning can change	removing one field or element can flip a status, a “not”, or a coordinate pair
Integrity and origin break	a label binding (4778) or a signature no longer matches a modified message — the receiver gets something the sender never sent
A violation is a signal	a malicious message says something about the sender or the channel; that deserves an alert and investigation, not silent cleaning

And “**mark as untrusted and pass it on**”? Not for a guard — that is how an IDS or monitoring system works. A guard’s job is precisely that nothing untrusted crosses. The only exception is **monitor / shadow mode**: the guard runs alongside an existing solution, logs its verdicts but does not block — for building evidence, not for operational use.

8.5 What happens to a rejected message

Default	hash plus verdict in the audit record; the original is discarded
Optional quarantine	for forensics, a copy of the original in an isolated, access-controlled, encrypted store on the ingress side , with a retention period. That copy never crosses the boundary and is never released automatically — release only by a person, back through the same guard path, rebuilt and inspected.

8.6 Three verdict states

Every message leaves exactly one record:

PASS	clean; only formatting noise dropped (category 1)
PASS-STRIPPED	passed after explicitly approved field removal (category 2); the removed fields are listed
REJECT	violation (category 3); whole message isolated — rule IDs, reason, hash, optional quarantine reference

This works identically for civilian and MIL use. Civilian customers will sometimes ask for “just clean it and pass it” for convenience. The answer is category 2: allowed, but **only** as an explicit strip rule in the contract — never as a default.

8.7 OWASP — a coverage map, not a rule port

It is reasonable to ask how the guard compares to OWASP and to a WAF’s managed rule set. The framing matters: the guard is **not** a WAF, so “we cover the WAF rules” is never the security claim — the

allow-list and schema conformance **are** the security property. Used correctly, the comparison is useful two ways: as a **coverage map** (recognised OWASP risk classes mapped to how the guard addresses each — injection → value

1. keyword rules; data exposure → field allow-list + releasability; XXE → the parser profile; integrity

failures → rebuild + CRC) for communication and gap-finding; and as a **test corpus** (OWASP CRS payloads fired at the guard as adversarial inputs — the hardened parser must fail-closed on every bomb; the allow-list must neutralise every injection). Never rules copied into the guard.

8.8 NIST 800-53 AC-4 — a control mapping

Where OWASP is a **test corpus**, NIST SP 800-53 Rev. 5 control **AC-4 (Information Flow Enforcement)** is a **control catalogue** — and it describes the guard almost component by component. Its enhancements (3)–(32) “primarily address cross-domain solution needs ... such as high-assurance guards.” Because NIST 800-53 is used far beyond defence (FedRAMP, government and critical-infrastructure compliance), this mapping is direct accreditation and compliance evidence for a civilian buyer:

AC-4(14)	filters requiring fully enumerated formats; validating schema structures	schema allow-list (Message Contract), field + value rules
AC-4(19)	policy filters on metadata	label guard (STANAG 4774/4778); ^ meta-data dropped
AC-4(25)	sanitise to minimise malicious content, command-and-control, malicious-code augmentation, steganography	canonical rebuild, no pass-through
AC-4(27)	redundant, independent filtering per data type	parser + rules + independent verifier; split guard
AC-4(28)	a linear content-filter pipeline	parse → caps → contract → label → rules → serialize
AC-4(30)	content filtering using multiple processes	sandboxed parser (<code>mode: component</code>); separate stages
AC-4(31)	prevent transfer of failed content to the receiving domain	fail-closed; reject → dead-letter; never re-pair

AC-4(31) is the verdict policy of this chapter stated as a control: **failed content never reaches the receiving domain**. Isolate — do not clean and pass.

PART III

Fielding It

Assurance, the path to market, and a buildable proof of concept.

9 Assurance, Deployment, and the Path to Market

The civilian guard's advantage is that it can be built and fielded now. This chapter is how: the assurance route, the hardware (mostly none), the roadmap, and the legal frame.

9.1 Assurance — climb in evidence first, then credentials

A serious buyer asks for evidence before certificates. So the guard climbs evidence first, and each step produces evidence the MIL track later reuses — the civilian route is the first half of the MIL route, not a detour.

1 — Engineering evidence	guard-profile fuzzing (coverage-guided, grammar-based, differential), OWASP/CRS payloads as a test corpus, reproducible native build, SBOM — the baseline any serious buyer asks for
2 — Independent pen test + code review	first external evidence
3 — Secure development process	ISO/IEC 27001 for the organisation; IEC 62443-4-1/-4-2 for OT buyers — opens critical-infrastructure
4 — NL: BSPA	Baseline Security Product Assessment (NBV), aimed at use up to Departementaal Vertrouwelijk / EU RESTRICTED — the first government credential
5 — Common Criteria	a modest level (e.g. EAL2–EAL4+ with a guard-specific security target) for international recognition
6 — → MIL edition	national high-assurance evaluation, NATO, and partners for the US NCDSMO route

(Verify current BSPA scope and procedure with the NBV/NLNCSA before planning; levels above are indicative.)

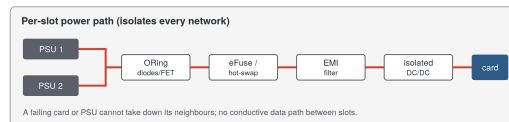
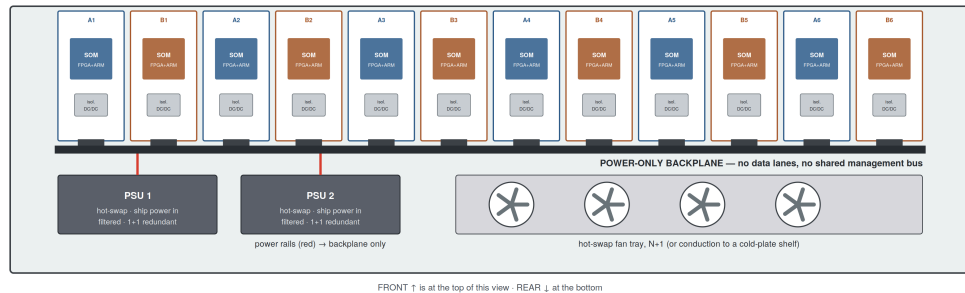
9.2 Hardware — mostly none

Civilian buyers run the guard as **software**; hardware climbs only where assurance or OT environments require it. No rugged defence hardware is needed for the civilian edition.

Software product	x86 or ARM server, VM or container (Open-M mode: component pod); GraalVM-native binary — shapes A and D
Diode-adjacent	a server beside a certified diode (Fox, Arbit, Owl, Waterfall) — shape B; the diode carries flow assurance
Demonstrator	two SoC-FPGA boards as a split guard — public formats only, already civilian
Appliance (later)	a small fanless box or a multi-guard chassis for OT and multi-flow sites

UTL-X Guard — 2U multi-guard chassis, 12 slots

Front view (to scale: 19-inch × 2U). Any slot accepts a SEND (A) or RECV (B) card; shown: 6 complete guards A1→B1 ... A6→B6.

**Top view (cover removed, schematic)****Design rules**

- Backplane carries power only (no Ethernet, PCIe, I²C/IPMB to cards).
- Each card: own seal, own mgmt fibre, own network fibre.
- Interlink A→B: front fibre jumper, keyed LINK OUT / LINK IN.
- Chassis monitoring (PSU, fans, temperature) stands alone, with no connection to any card.
- Shielded card cans + filtered power per slot for TEMPEST.
- 12 full guards = 24 cards = two of these chassis (4U).

Figure — Illustrative concept of a 2U, 12-slot UTL-X multi-guard chassis with power-only redundant backplane (not an existing product; card internals schematic).

Figure 3: Illustrative appliance concept for multi-flow / OT sites: a 2U chassis in which every slot is a complete, independent guard on a power-only backplane — no shared data lanes. A later option, not needed for the software product.



Figure 4: Hot-swap serviceability of the appliance concept: a single guard card withdrawn from the front. All data enters and leaves on the card itself; the power-only backplane means removing or inserting a card never bridges two networks. Slots mix send and receive cards.

The split guard itself — a buildable proof of concept on two boards — is Chapter 10.

9.3 Roadmap

A phased programme, each phase a usable increment:

0 — Guard core	guard-profile parsers, canonical serializer, the seven rule categories, audit, dead-letter, no-pass-through — a software guard for JSON/XML/CSV/YAML/OData
1 — Foundation	BINF, streaming runtime, transports, tests — a binary-capable engine
2 — Tier A packs	AIS/NMEA, KLV, EDIFACT, CoT, CISE, geometry — coast-guard and dual-use scope
3 — Tier B packs	ASTERIX, RIS, IVEF, S-100 products, DIS, C2SIM, Cospas-Sarsat — full open scope
4 — Productisation	contract/policy UI/CLI, monitoring, packaging, docs — a sellable product
5 — Assurance	fuzzing campaign, pen test, 62443/BSPA preparation

Order inside the phases follows the chosen wedge: a CISE-first wedge pulls CISE and AIS forward; an OT-first wedge prioritises guard core plus shape-B integration with one diode vendor. The whole programme is a **bounded effort for a small team** — far short of the multi-year accreditation campaign the MIL edition requires. (Detailed effort estimates are held separately, for investment discussions.)

9.4 Legal, licensing, export

Licensing	open-core: engine and public packs under AGPL-3.0; guard productisation, certified builds, support and policy tooling commercial. Offer a commercial licence where AGPL is unacceptable to a buyer.
------------------	---

Export control	open standards remove the restricted-standards problem; cryptographic functions (TLS, interlink HMAC, label signatures) may still fall under EU dual-use rules — far lighter than ITAR, but check before the first sale outside the EU
No restricted content	the engine stays content-free; restricted form-classes never enter the civilian repository

9.5 Bottom line

A civilian, open-standards guard can be built and fielded **now**, without a sponsor or restricted access, on the same engine and architecture as the MIL guard. It creates customers, references and assurance evidence — exactly what the MIL track lacks today. Build it first, keep the six non-negotiables and the verdict policy uncompromised, and the MIL edition becomes an extension rather than a separate bet.

10 Proof of Concept — the Split Guard

Everything in this book is a claim until it is built. This chapter is a concrete, buildable proof of concept — deliberately small, on **public hardware and public data** — that exercises the whole civilian guard story at once, followed by an honest statement of where a demonstrator stops and a funded step begins. Because nothing in the civilian edition is restricted, the demonstrator needs no sponsor and no clearance: it can be built now.

10.1 The demonstrator: a split guard on two boards

The guard's strongest architecture is **transform** → **verify**: software turns complex input into a simple, strongly-typed form; independent logic verifies that form; the delivered message is rebuilt only from verified data. The proof of concept makes this physical by **splitting the guard across two boards** joined by a one-way link — the smallest version of the two-owner split guard (shape D).

Board A — ingest / transform	receives several formats; runs UTL-X on the Arm cores to parse → UDM → validate.* → re-serialize to a simple, strongly-typed form ; uses the FPGA fabric to decode a binary sample (BINF) and enforce structural caps
Link — protocol break	a fibre SFP+ connection carrying only the simple form; made physically one-way it emulates the diode / protocol break — no TCP/IP state crosses
Board B — verify / egress	FPGA fabric verifies the simple form against the form-class / schema; the Arm side rebuilds and forwards only verified data; anything failing → dead-letter (fail-closed)

Two identical **Xilinx ZCU106** boards (Zynq UltraScale+ MPSoC: Arm cores + FPGA fabric on one chip) are the split pair.

Board note (honest). The cheaper **ZCU104** or **Kria KR260** are an equally good fit; two ZCU106 are used here simply as available, identical SoC-FPGA boards. The **split** — transform on one chip, verify on the other — is the point, not the exact board.

10.2 Architecture

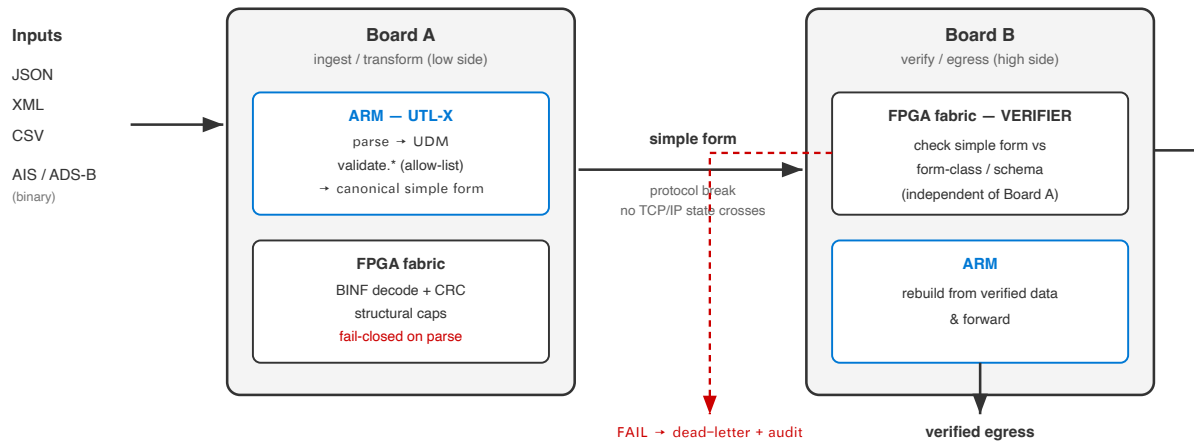
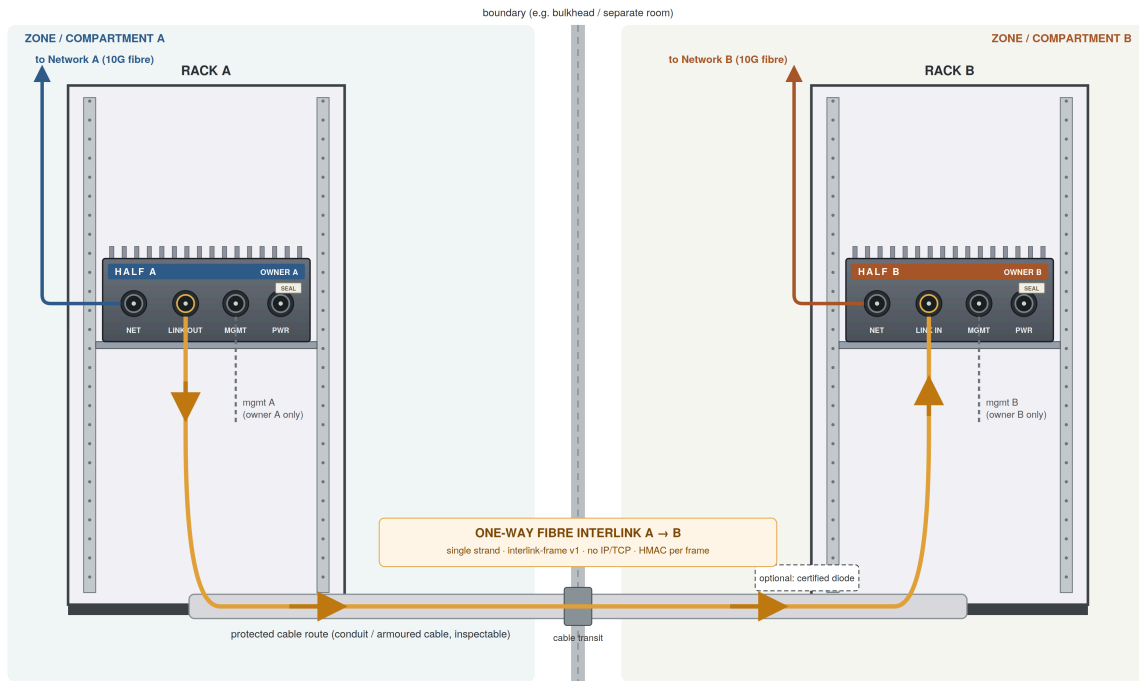


Figure 5: The split guard: Board A transforms any input format into a simple, strongly-typed form; a protocol-break link carries only that form; Board B independently verifies it and forwards only verified data. Anything failing is dead-lettered (fail-closed).

The Arm side runs the existing GraalVM-native UTL-X binary (`linux-aarch64`) unchanged — no new port. In deployment this same split becomes the two-owner guard pair, meeting over fibre (multimode in a building, single-mode or dark fibre between sites), with one-way enforcement carried in the optics:



Design notes

- LINK OUT (A) has only a transmit terminus, LINK IN (B) only a receive terminus — one-way is physical; differently keyed connector.
- Distance: 10GBASE-SR ≈ 300 m (OM3) / 400 m (OM4); 10GBASE-LR up to 10 km single-mode. Pull spare fibres, leave them unterminated.
- The HMAC protects integrity, not confidentiality; route the cable through protected space, or add an approved link encryptor.
- Each rack has its own power and ground; fibre carries no current, so no ground loop between compartments.

Figure — Split guard with each half in its own rack and zone, joined only by a visible one-way fibre interlink (illustrative concept).

Figure 6: The split guard (shape D) fielded: each half in its own rack, zone and ownership (A / B), enforcing its own policy and meeting only over a one-way fibre interlink. The proof of concept builds exactly this on two SoC-FPGA boards with public formats — already civilian.

10.3 What it demonstrates

- **Universal parsing:** JSON, XML, CSV and a **binary** AIS/ADS-B frame all reduced to one UDM.
- **One rule set, many formats:** the same `validate.*` block (structural caps, schema conformance, value ranges, labelling) applied across all of them.
- **BINF in fabric:** a fixed binary frame decoded + CRC-checked in FPGA logic at the edge.
- **Allow-list / fail-closed:** only conforming, approved messages reach Board B; the rest are dead-lettered.
- **Transform → verify:** Board A transforms; Board B **independently** verifies a simple form; delivery is rebuilt from verified data.
- **Protocol break:** no transport state crosses the link; two-way traffic would be two separate one-way flows.

10.4 Success criteria (indicative)

1. The **same** rule block passes a valid JSON position report and a valid AIS frame, and produces identical downstream output (e.g. CoT / JSON).
2. An over-limit message (depth/size bomb), a message with a forbidden field, and a message failing a label check are each **blocked, fail-closed**, with an audit record (REJECT).
3. A public AIS frame is decoded in Board A's fabric and mapped to CoT / JSON.
4. A **tampered** simple form is **rejected by Board B's verifier** (not by Board A) — proving the independence of the verify stage.
5. End-to-end latency and a rough throughput figure are measured — **indicative only**.

10.5 Scope and effort

- **Formats:** public only — JSON, XML, CSV, and public **AIS/ADS-B**.
- **Rules:** a small policy from the seven categories of Chapter 8, bound to one approved contract.
- **Effort:** order of weeks, one engineer familiar with UTL-X plus a Vitis/HLS starting point; reuses the existing native binary and the eval-board toolchain.
- **Indicative hardware:** two ZCU106 (or, cheaper, two ZCU104 / KR260), plus a host PC.

10.6 What this PoC deliberately does NOT do

A capability and architecture demonstrator, **not** a product. It excludes operational throughput, a rugged/appliance form, redundancy; a certified crypto / data diode (the protocol break is **emulated**); and any formal assurance (no IEC 62443, BSPA, or Common Criteria yet). Note what it does **not** need, which the MIL edition does: **no TEMPEST, no emission-security facility, no NATO/NCDSMO campaign**. The civilian scale-up is a far smaller climb.

10.7 The investment case (civilian)

Moving from this demonstrator to a sellable civilian product is **investment, but not a mountain** — and, critically, it is funded by the first wedge customer rather than by a multi-year accreditation programme. The scale-up is three things, none of them defence hardware:

Assurance	the evidence ladder of Chapter 9 — fuzzing campaign, independent pen test, then IEC 62443-4-2 (OT) or BSPA (government) as the first credential the wedge customer requires
Productisation	contract and policy tooling (UI/CLI), monitoring, packaging, docs — turning the engine into something an operator runs
Scale & integration	line-rate throughput where needed, and a certified diode partner for shape B (the diode carries flow assurance; UTL-X carries content) — not bespoke silicon

Contrast the MIL edition, whose scale-up **is** a mountain: TEMPEST, certified high-assurance crypto, a certified verifier, rugged SOSA hardware, and a multi-year national/NATO accreditation campaign. The civilian product reaches revenue first, on software, and **funds** that later climb.

10.8 Next steps

1. **Build the demonstrator** (weeks, public hardware and data) and show criteria 1–5.
2. **Pick the wedge** — coast guard / CISE, critical-infrastructure OT, or government data sharing — for a low-assurance first deployment with immediate value.
3. **Engage** the assurance route (62443 or BSPA) and, for shape B, a diode vendor — early.
4. **Scope the funded programme** against the civilian investment case above — then let its revenue and references fund the MIL edition.

11 Appendix A: UTL-X Quick Reference

A *syntax cheat-sheet*. For the complete reference see UTL-X: One Language, All Formats.

11.1 File structure

```
%utlx 1.1           // a guard validates, so 1.1; 1.0 for a pure transform
input json
output json
---
{ transformation body }
```

11.2 Format headers

input json / xml / csv / yaml	self-describing text formats
input track binf {definition: "t.def"}	binary via a form-class (Ch 6)
input ais	named alias → binf + registered form-class
output xml	output format (same as input, or different)

11.3 Access

\$in.name (.)	data
\$in.El.@id (@)	attribute
\$in.x^crcValid (^)	metadata about a node (incl. the label)

11.4 Validation (1.1)

```
validate.inRange($m.lat, -90, 90)
validate.oneOf($m.status, ["OK", "WARN", "FAIL"])
validate.conformsTo($m, "contracts/approved/x:1.0.0")
validate.assert($m.stale > $m.time, "stale after time")
```

11.5 The guard, in one line

```
untrusted bytes → hardened parse → UDM → validate.* rules → canonical serialize →
egress
└── any failure → isolation + audit +
alert
```

11.6 The non-negotiables

no pass-through · allow-list / fail-closed · pure 1.0/1.1 only (never ai.*) · canonical low-fidelity output
· every verdict auditable · UDM only sees what it models.

11.7 The verdict policy

rebuild what is clean, reject what is wrong — never repair · three categories (formatting noise / policy-strip / violation) · three verdicts (PASS / PASS-STRIPPED / REJECT).

12 Appendix B: Open-Format Catalogue

Every format in the civilian scope, by encoding, with the mechanism it needs. All are publicly specified; nothing here is restricted. “Mechanism” is what the engine does to read it.

12.1 Text-structured → native readers (mapping only)

JSON / YAML	native — the modern-IT side of most bridges
XML (incl. GML)	native — CISE, IVEF, S-100 products, CoT, C2SIM, labels
CSV	native — OT historian exports, tabular feeds
OData	native — government data services
Protobuf / Avro	native (schema) — TAK Protobuf and data pipelines

12.2 Schema / metadata → native

XSD, JSON Schema	structural conformance (<code>validate.conformsTo</code>)
OData EDMX, Table Schema	service and tabular contracts

12.3 Binary → BINF codec + form-class

AIS (NMEA 0183)	6-bit-packed binary; the ideal first binary pack
ADS-B (1090ES)	fixed 112-bit frames + 24-bit CRC
ASTERIX	binary, FSPEC presence bitmaps — the variable-structure stress test
MISB KLV ST 0601	KLV byte-structured, self-tagged — FMV metadata
DIS	binary PDUs — simulation

12.4 EDI / delimited → EDIFACT-class reader

UN/EDIFACT	ports, customs, logistics
RIS (ERI, NtS, Inland AIS)	inland shipping — routes by sub-format

12.5 Code systems / labels

APP-6 / MIL-STD-2525 (SIDC)	symbol identity codes — a lookup, not a codec
STANAG 4774 / 4778 (public)	confidentiality label syntax + binding — XML + <code>validate.*</code>

12.6 Out of scope (MIL edition)

Link 16 J-series, JREAP-C, VMF, Link 22, and any standard of unclear release status — separately governed packs, never in the civilian repository.

13 Appendix C: Sources & Further Reading

This book synthesises the UTL-X design documents and public standards. The design docs carry their own sourced references.

13.1 UTL-X design documents

utl-x/docs/architecture/civilian-guard.md	the civilian-first strategy: product, markets, assurance route, roadmap, risks — the basis of this book
utlx-mil/docs/UDM-content-guard.md	the guard architecture: CDS/CDR framing, threat model, pipeline, deployment shapes, verdict policy, non-negotiables
utlx-mil/docs/guard-rule-library.md	the seven <code>validate.*</code> rule categories + OWASP coverage map
utlx-mil/docs/guard-parser-profile.md	the hardened read profile + canonical low-fidelity serializer
utlx-mil/docs/BINF-bit-level-binary-format.md	the form-class / bit-level codec
utl-x/docs/proposals/utlx-1_1-semantic-validation.md	the 1.1 <code>validate.*</code> namespace and <code>ValidationResult</code>
utl-x/docs/architecture/utlx-test-corpora.md	the test doctrine for hardening the parsers and serializers

13.2 Companion books

<i>UTL-X MIL — The Content Guard</i>	the defence edition: restricted packs, accreditation, rugged hardware — same engine and architecture
<i>UTL-X: One Language, All Formats</i>	the complete language specification
<i>UTLXe on Azure</i>	running the engine as a managed cloud component with Dapr

13.3 External standards & guidance (verify against primary sources)

- **Assurance:** NL BSPA (NBV/NLNCSA); IEC 62443-4-1/-4-2 (OT secure development/components); ISO/IEC 27001; Common Criteria.
- **Regulation:** NIS2 and national implementations; EU dual-use Regulation (EU) 2021/821.
- **Labels:** STANAG 4774 / 4778 (public).
- **Open formats:** EU CISE; ITU-R M.1371 (AIS); EUROCONTROL ASTERIX; RTCA DO-260B (ADS-B); MISB ST 0601 (KLV); MITRE Cursor-on-Target; UN/EDIFACT; IHO S-100; IALA IVEF; Cospas-Sarsat.
- **Security benchmark:** OWASP Top 10 / API Security Top 10 / Core Rule Set — used as coverage map and test corpus, never as a rule port.

13.4 Cross-domain literature & standards basis

The guard concept, its taxonomy and its controls are well established — the classification and the AC-4 mapping in this book rest on these, not on our own definitions:

- R. Smith, **Constructing a High Assurance Mail Guard**, 17th National Computer Security Conference (1994) — the store-and-forward mail guard: address allow-lists, label checks, attachment filtering; assurance “even under attack and failure”.
- Sundaravarathan et al., **Cross-Domain Solutions (CDS): A Comprehensive Survey**, IEEE Access (2024) — the access / transfer / MLS taxonomy; uni- vs bidirectional transfer.

- **NIST SP 800-53 Rev. 5**, control AC-4 (Information Flow Enforcement) — the CDS / high-assurance-guard control enhancements (Chapter 8).
- US DoDI 8540.01 (CDS types) & NSA NCDSMO; CNSSI 1253 Cross Domain Solution Overlay; UK NCSC cross-domain principles & 2026 guidance; NZ NZISM; AU ACSC introduction to cross-domain solutions.

UTL-X Guard — The Civilian Edition · Exploratory edition, 2026 · MIL-ready by design, proven in civilian service.